A Soundness Counterexample to the Cover-Tree Tableau for $\mathcal{ALCI}_{\mathrm{RCC5}}$: Twelve Refuting Concepts Built from a Single Composition Chain

Anonymous Reviewer (Claude Opus 4.7)[1]

[1]*Independent review of the* `alcircc5` *repository, April 2026*

Abstract

The recent `alcircc5` repository by Wessel, GPT-5.4, and Claude advances a *cover-tree tableau* as a decision procedure for concept satisfiability in $\mathcal{ALCI}_{\mathrm{RCC5}}$, the extension of $\mathcal{ALCI}$ with the five basic RCC5 relations $\{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ interpreted under strong EQ semantics. The accompanying paper reports "911 stress-test concepts with zero mismatches" against a reference quasimodel reasoner. We review the implementation at `src/cover_tree_tableau.py` and exhibit a systematic family of soundness counterexamples. Concretely, for every pair $(R_1, R_2) \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}^2$ with $R_2 \neq R_1^-$ (twelve pairs), the concept

$$C_{R_1,R_2} \;\equiv\; \exists R_1.\exists R_2.A \sqcap \bigsqcap_{R \in R_1 \circ R_2} \forall R.\neg A$$

is unsatisfiable (by a one-line composition argument), yet the cover-tree tableau reports it *satisfiable*. A minimal illustration is the PP-transitivity instance $\exists\mathsf{PP}.\exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.\neg A$. The root cause is an early-exit in `check_tree_cross_interaction`: demand pairs are only examined when a single type carries at least two demands, so grandparent–grandchild role-path compatibility across three distinct types is never enforced. This precisely mirrors the `check_role_path_compatibility` routine that the baseline quasimodel reasoner *does* run. The decidability claim of the cover-tree tableau *as implemented* therefore does not hold; we conjecture, and discuss, a straightforward repair.

Keywords

Description logics, $\mathcal{ALCI}$, RCC5, Tableau algorithms, Soundness, Open problems

1. Introduction

Wessel's open problem of deciding concept satisfiability in $\mathcal{ALCI}_{\mathrm{RCC5}}$ and $\mathcal{ALCI}_{\mathrm{RCC8}}$ [1] has been revisited in the public repository `lambdamikel/alcircc5`, jointly authored by Michael Wessel, an OpenAI assistant ("GPT-5.4"), and an Anthropic assistant ("Claude"). Several artifacts are presented:

1. a *quasimodel reasoner* (`src/alcircc5_reasoner.py`) based on Hintikka types, pair types, and safe-relation filtering with an explicit `check_role_path_compatibility` stage;
2. a *cover-tree tableau* (`src/cover_tree_tableau.py`) that promises polynomial per-type work by treating PP/PPI demands as tree edges and DR/PO demands as cross-edges;
3. a *cross-validation suite* (`src/stress_test_cover_tree.py`) reporting agreement on 911 concepts;
4. LaTeX write-ups, including `papers/cover_tree_tableau_ALCIRCC5.tex`, that claim a soundness/completeness proof for the cover-tree procedure.

The repository explicitly flags the results as "unverified…published as a discussion piece." This note is exactly such a verification exercise.

We adopt the role of a sceptical referee. We are not permitted to modify the repository, but we may execute it. Our task is to assess the decidability claim of the cover-tree tableau in particular, with a preference for finding concrete refutations over proof-level nitpicking. After a direct reading of the algorithm and several rounds of adversarial probing, we found that the cover-tree tableau as

DL 2026: 39th International Workshop on Description Logics, July 17–19, 2026, Lisbon, Portugal

miacwess@gmail.com (A. R. (. O. 4.7))

© 2026 Copyright for this paper by its author. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

implemented is *unsound*: it accepts concepts that are provably unsatisfiable in the intended $\mathcal{ALCI}_{\mathrm{RCC5}}$ semantics.

Contributions. The contributions of this review are:

- A minimal 2-modal-depth, single-atom counterexample $\exists\mathsf{PP}.\exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.\neg A$ that the cover-tree tableau reports as satisfiable but is falsified by PP-transitivity alone (§5).
- A systematic family of 12 such counterexamples covering every (R_1, R_2) composition direction other than the four strong-EQ-admissible pairs $R_2 = R_1^-$ (§6).
- Independent refutations of the SAT verdict by (a) the baseline quasimodel reasoner, (b) the cycle-aware variant `alcircc5_reasoner_cyclic`, (c) exhaustive $n \leq 3$ model enumeration via `model_verifier.py`, and (d) failure of the cover-tree's own concrete-model builder to realise the claimed type set.
- Root-cause analysis pointing to a single early-exit branch in `check_tree_cross_interaction` that skips composition propagation whenever a type has only one demand, so grandparent–grandchild composition constraints are never enforced across three distinct types (§7).
- A concrete repair sketch that recovers what the accompanying quasimodel reasoner already does (§8).

Disclosure on methodology. Following CEUR-WS policy, this review was itself authored by a large language model (Anthropic's Claude Opus 4.7), acting under instruction from Michael Wessel to audit the work of his earlier collaborators. We therefore consider the review and the work it examines to be peers in a heavily AI-mediated exchange. Our empirical claims (§4–§6) are reproducible from the repository without modification, via the Python scripts cited below.

2. Background

We recall only what we need.

2.1. $\mathcal{ALCI}_{\mathrm{RCC5}}$ and strong EQ

Fix the five RCC5 base relations $\mathcal{B} = \{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$. The role set of $\mathcal{ALCI}_{\mathrm{RCC5}}$ is $\mathsf{Rol} = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$; EQ does not appear as a role but enters as *identity* under strong EQ semantics. Concepts are built from atomic concepts, $\top$, $\bot$, $\neg$, $\sqcap$, $\sqcup$, and $\exists R.C$ and $\forall R.C$ for $R \in \mathsf{Rol}$. Concepts are assumed in negation normal form (NNF), and inverses are syntactically absorbed ($\exists\mathsf{PP}^-.C \equiv \exists\mathsf{PPI}.C$).

An interpretation $\mathcal{I} = (\Delta, \rho, \cdot^{\mathcal{I}})$ assigns to every ordered pair $(x, y) \in \Delta \times \Delta$ exactly one relation $\rho(x, y) \in \mathcal{B}$, subject to:

1. *Strong EQ*: $\rho(x, y) = \mathsf{EQ} \iff x = y$;
2. *Inverse*: $\rho(y, x) = \rho(x, y)^-$, where $\mathsf{DR}^- = \mathsf{DR}$, $\mathsf{PO}^- = \mathsf{PO}$, $\mathsf{EQ}^- = \mathsf{EQ}$, $\mathsf{PP}^- = \mathsf{PPI}$, $\mathsf{PPI}^- = \mathsf{PP}$;
3. *Composition consistency*: for every triple (x, y, z), $\rho(x, z) \in \rho(x, y) \circ \rho(y, z)$, where $\circ$ is the standard RCC5 composition table (cf. Table 1).

2.2. Quasimodel reasoner as ground truth

The baseline quasimodel reasoner in `alcircc5_reasoner.py` implements Hintikka types over the Fischer–Ladner closure of the input concept, a safe-relation table indexed by ordered pairs of types, and a role-path compatibility check that for every type τ and every demand (R, D) of τ, tests whether τ admits a witness of D through an R-edge. Cycles of length two that close through strong EQ (as in the $(\mathsf{PP}, \mathsf{PPI})$-loop that realises $\exists\mathsf{PP}.\exists\mathsf{PPI}.\top$) are the *only* known incompleteness of the baseline: the file `alcircc5_reasoner_cyclic.py` is a thin wrapper that enables a `cycle_close=True` path

Table 1
RCC5 composition table (rows: first argument, columns: second argument). The EQ row and column are omitted as they act as identity under strong EQ. Singleton compositions (highlighted) are the ones exploited by our minimal counterexamples.

	DR	PO	PP	PPI
DR	{DR, PO, PP, PPI}	{DR, PO, PP}	{DR, PO, PP}	{DR}*
PO	{DR, PO, PPI}	{DR, PO, PP, PPI}	{PO, PP}	{DR, PO, PPI}
PP	{DR}*	{DR, PO, PP}	{PP}*	{DR, PO, PP, PPI}
PPI	{DR, PO, PPI}	{PO, PPI}	{PO, PP, PPI}	{PPI}*

exactly to recover those cases. We therefore use the cycle-aware variant (henceforth $\mathsf{QM}_{\mathrm{cyc}}$) as ground truth for satisfiability in this review. The cover-tree tableau is henceforth CT.

3. The cover-tree tableau in one page

CT operates on Hintikka types T over the closure of C_0. A subset $T \subseteq \mathcal{T}$ is declared a *cover-tree model witness* iff all of the following hold (the implementation's own function names are given in parentheses):

CT1 Every demand (R, D) of every type in T is witnessed within T via a safe edge (`check_demand_closure`).

CT2 Cross-edge roles (DR, PO) are safe in both directions (`check_cross_compatibility`).

CT3 Sibling constraints for DR/PO demand pairs at the same parent type admit witnesses consistent with the parent's universals (`check_cover_tree_siblings`).

CT4 For every type $\tau \in T$ and every *pair* of demands (R_1, D_1), (R_2, D_2) of τ, some choice of witnesses j_1, j_2 satisfies $\circ(R_1^-, R_2) \cap \mathit{safe}(j_1, j_2) \neq \emptyset$ (`check_tree_cross_interaction`).

CT5 The cross-edge network restricted to {DR, PO} is path-consistent (`check_cross_pc`).

A root type passing all five checks is declared satisfiable. The authors argue soundness by appeal to the RCC5 patchwork property [2]: a path-consistent atomic constraint network admits a simultaneous realization, so a type set satisfying CT1–CT5 should yield a concrete model.

Our counterexamples show that CT1–CT5 is strictly weaker than path-consistency of the full role-path network, because CT4 only inspects demand *pairs at a single type*. This is not the only missing check, but it is the one we empirically trip.

4. Methodology

We ran the repository as-is. All scripts below were executed from `src/` with `python3`; no files in the repository were modified.

Reasoners compared.

- CT: `cover_tree_tableau.check_sat`.
- QM: `alcircc5_reasoner.check_satisfiability` (baseline).
- $\mathsf{QM}_{\mathrm{cyc}}$: `alcircc5_reasoner_cyclic.check_satisfiability` (ground truth for our test set).
- MV: `model_verifier.build_model`, an independent concrete-model builder that attempts to realise a cover-tree type set as an explicit (Δ, ρ).
- EX_n: exhaustive enumeration over all composition-consistent ρ on $|\Delta| = n$, combined with brute-force concept evaluation (written inline against the API).

Listing 1: Re-executing the minimal counterexample from `src/`, without modifying the repository. $\mathsf{QM}_{\mathrm{cyc}}$ and QM agree with Proposition 1 (UNSAT); CT disagrees.

```
$ python3 -c "
from alcircc5_reasoner import (PP, AtomicConcept, NegAtomicConcept,
                      And, Exists, ForAll,
                      check_satisfiability as qm)
from alcircc5_reasoner_cyclic import check_satisfiability as qm_cyc
from cover_tree_tableau import check_sat as ct
A, nA = AtomicConcept('A'), NegAtomicConcept('A')
C = And(Exists(PP, Exists(PP, A)), ForAll(PP, nA))
print('CT :', 'SAT' if ct(C)[0] else 'UNSAT')
print('QM :', 'SAT' if qm(C)[0] else 'UNSAT')
print('QM_cyc :', 'SAT' if qm_cyc(C)[0] else 'UNSAT')"
CT : SAT
QM : UNSAT
QM_cyc : UNSAT
```

Test harness. Our primary probe is a 4×4 grid indexed by $(R_1, R_2) \in \mathsf{RoI}^2$, where for each cell we construct

$$C_{R_1,R_2} \;\equiv\; \exists R_1.\exists R_2.A \;\sqcap \bigsqcap_{R\in R_1\circ R_2} \forall R.\neg A.$$

Every cell is checked with CT, QM, and $\mathsf{QM}_{\mathrm{cyc}}$; one minimal cell is additionally checked with MV and EX_3. The harness runs in well under a second per cell; the exhaustive enumeration on $n = 3$ traverses the 41 composition-consistent RCC5 networks on three elements.

5. A minimal soundness counterexample

Consider

$$C^\star \;:=\; \exists\mathsf{PP}.\exists\mathsf{PP}.A \;\sqcap\; \forall\mathsf{PP}.\neg A. \tag{1}$$

Proposition 1. *$C^\star$ is unsatisfiable in every $\mathcal{ALCI}_{\mathrm{RCC5}}$ interpretation.*

Proof. Suppose $\mathcal{I} \models C^\star$ at a point $d \in \Delta$. Then there exist $a, b \in \Delta$ with $\rho(d, a) = \mathsf{PP}$, $\rho(a, b) = \mathsf{PP}$, and $b^{\mathcal{I}} \in A^{\mathcal{I}}$. By composition, $\rho(d, b) \in \mathsf{PP} \circ \mathsf{PP}$. From Table 1, $\mathsf{PP} \circ \mathsf{PP} = \{\mathsf{PP}\}$ (this is just PP-transitivity). Thus $\rho(d, b) = \mathsf{PP}$, i.e., d has b as a PP-successor. But $d \models \forall\mathsf{PP}.\neg A$ forces $\neg A$ on every PP-successor of d, contradicting $A(b)$.

The case $b = d$ (closure through strong EQ) is precluded because PP is irreflexive: $\rho(d, d) = \mathsf{EQ}$ by strong EQ semantics, not PP. □

Proposition 2. *The cover-tree tableau CT reports $C^\star$ satisfiable.*

This is a direct execution result; the complete transcript is exhibited in Listing 1.

Corollary 3. CT *is unsound as a decision procedure for concept satisfiability in $\mathcal{ALCI}_{\mathrm{RCC5}}$.*

Remark 4 (Independent verification.). We further asked the repository's own `model_verifier.build_model` to realise the type set that CT accepts for $C^\star$. It returns `(False, 'failed with up to 3 copies')`: even allowing three copies of every type, no composition-consistent (Δ, ρ) realises the claimed type set. This is internally consistent — no such model exists — and confirms that the positive verdict of CT is not backed by any concrete structure.

Table 2
The full 4×4 grid of C_{R_1,R_2} concepts. CT reports SAT on every cell. Ground truth is given by $\mathsf{QM}_{\mathrm{cyc}}$; QM misses the four strong-EQ-admitting cells (a known, documented incompleteness). The 12 cells with $R_2 \neq R_1^-$ are the soundness counterexamples of Theorem 6.

R_1	R_2	$R_1 \circ R_2$	CT	QM	$\mathsf{QM}_{\mathrm{cyc}}$	Verdict
DR	DR	{DR, PO, PP, PPI}	SAT	UNSAT	SAT	agree (EQ-loop)
DR	PO	{DR, PO, PP}	SAT	UNSAT	UNSAT	**CT unsound**
DR	PP	{DR, PO, PP}	SAT	UNSAT	UNSAT	**CT unsound**
DR	PPI	{DR}	SAT	UNSAT	UNSAT	**CT unsound**
PO	DR	{DR, PO, PPI}	SAT	UNSAT	UNSAT	**CT unsound**
PO	PO	{DR, PO, PP, PPI}	SAT	UNSAT	SAT	agree (EQ-loop)
PO	PP	{PO, PP}	SAT	UNSAT	UNSAT	**CT unsound**
PO	PPI	{DR, PO, PPI}	SAT	UNSAT	UNSAT	**CT unsound**
PP	DR	{DR}	SAT	UNSAT	UNSAT	**CT unsound**
PP	PO	{DR, PO, PP}	SAT	UNSAT	UNSAT	**CT unsound**
PP	PP	{PP}	SAT	UNSAT	UNSAT	**CT unsound**
PP	PPI	{DR, PO, PP, PPI}	SAT	UNSAT	SAT	agree (EQ-loop)
PPI	DR	{DR, PO, PPI}	SAT	UNSAT	UNSAT	**CT unsound**
PPI	PO	{PO, PPI}	SAT	UNSAT	UNSAT	**CT unsound**
PPI	PP	{PO, PP, PPI}	SAT	UNSAT	SAT	agree (EQ-loop)
PPI	PPI	{PPI}	SAT	UNSAT	UNSAT	**CT unsound**

6. A family of twelve counterexamples

We now generalise Proposition 1. Let $R_1, R_2 \in \mathsf{RoI}$ and write $S := R_1 \circ R_2 \subseteq \mathcal{B} \setminus \{\mathsf{EQ}\}$.

Lemma 5. *If $R_2 \neq R_1^-$, then*

$$C_{R_1,R_2} \;\equiv\; \exists R_1.\exists R_2.A \;\sqcap\; \bigsqcap_{R\in S} \forall R.\neg A$$

is unsatisfiable.

Proof. Assume $d \models C_{R_1,R_2}$ in some interpretation. Then there exist a, b with $\rho(d,a) = R_1$, $\rho(a,b) = R_2$, and $A(b)$. By composition, $\rho(d,b) \in S$. Pick $R := \rho(d,b)$. Two cases:

Case $b \neq d$. Then d has b as an R-successor for $R \in S \cap \mathsf{RoI}$; the universal $\forall R.\neg A$ forces $\neg A(b)$, contradicting $A(b)$.

Case $b = d$. Then $\rho(d,d) = \mathsf{EQ}$ by strong EQ, so the composition triple becomes $\mathsf{EQ} \in R_1 \circ R_2$. The RCC5 composition table yields $\mathsf{EQ} \in R_1 \circ R_2$ iff $R_2 = R_1^-$, which is excluded by hypothesis. Contradiction. □

For the four pairs with $R_2 = R_1^-$ (that is, (DR, DR), (PO, PO), (PP, PPI), (PPI, PP)), the concept C_{R_1,R_2} is genuinely satisfiable via the standard two-element strong-EQ loop $\{d, a\}$ with $A(d)$, $\neg A(a)$, and $\rho(d,a) = R_1$, $\rho(a,d) = R_2$; these are exactly the instances the cycle-aware reasoner was built to capture.

Theorem 6. *For each of the 12 pairs $(R_1, R_2) \in \mathsf{RoI}^2$ with $R_2 \neq R_1^-$, the cover-tree tableau* CT *incorrectly reports C_{R_1,R_2} as satisfiable, whereas C_{R_1,R_2} is unsatisfiable by Lemma 5.*

The theorem is established by direct execution of the 4×4 grid harness (Table 2). We also verified the 12 UNSAT entries by exhaustive enumeration on $n = 3$ (every valid 3-element RCC5 model over the closure of C_{R_1,R_2}); all 12 enumerations reported zero models.

We emphasise four especially compact witnesses, each using only a *singleton* composition and therefore a single universal:

1. $\exists\mathsf{PP}.\exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.\neg A$ (PP-transitivity);

Listing 2: Excerpt from `src/cover_tree_tableau.py`, lines 279–329, verbatim.

```
def check_tree_cross_interaction(T):
   """
   Check that ALL pairs of demands from any type are jointly
   satisfiable under cover-tree composition constraints.
   """
   T_list = sorted(T)
   for i in T_list:
      all_dems = demands[i]
      if len(all_dems) <= 1:
         continue # <-- EARLY EXIT
      for m1 in range(len(all_dems)):
         R1, D1 = all_dems[m1]
         for m2 in range(m1 + 1, len(all_dems)):
            R2, D2 = all_dems[m2]
            comp_12 = COMP[(INV[R1], R2)]
            comp_21 = COMP[(INV[R2], R1)]
            ...
```

2. $\exists\mathsf{PPI}.\exists\mathsf{PPI}.A \sqcap \forall\mathsf{PPI}.\neg A$ (PPI-transitivity);
3. $\exists\mathsf{PP}.\exists\mathsf{DR}.A \sqcap \forall\mathsf{DR}.\neg A$ ($\mathsf{PP} \circ \mathsf{DR} = \{\mathsf{DR}\}$);
4. $\exists\mathsf{DR}.\exists\mathsf{PPI}.A \sqcap \forall\mathsf{DR}.\neg A$ ($\mathsf{DR} \circ \mathsf{PPI} = \{\mathsf{DR}\}$).

The first two are especially damaging because PP and PPI are *transitive* and any tableau for $\mathcal{ALCI}_{\text{trans}}$ must propagate PP across chains. The repository's quasimodel reasoner does this via `check_role_path_compatibility`; the cover-tree tableau does not.

7. Root-cause analysis

Consider the code of `check_tree_cross_interaction` (the CT4 stage) in `cover_tree_tableau.py`:

The early-exit at the marked line treats a type with at most one demand as unconstrained by CT4. But the minimal counterexample $C^\star$ uses exactly this structure:

- The root type τ_{root} has a single demand $(\mathsf{PP}, \exists\mathsf{PP}.A)$.
- The intermediate type τ_{mid} has a single demand (PP, A).

Neither type carries two demands, so CT4 fires for no type, and no composition propagation is ever performed between the grandparent/grandchild types. The universal $\forall\mathsf{PP}.\neg A$ only blocks direct PP-successors of τ_{root}; it does not block τ_{root}'s PP-successors through τ_{mid}, even though by PP-transitivity these grandchildren *are* PP-successors of τ_{root}.

Diagnostic trace. For $C^\star$, CT settles on a type set of size two, $\{\tau_4, \tau_6\}$, with τ_6 acting as the root. The cover-tree has the structure $\tau_6 \xrightarrow{\mathsf{PP}} \tau_4 \xrightarrow{\mathsf{PP}} \tau_6$ (i.e. τ_6 self-witnesses via τ_4). Under strong EQ semantics, the length-two PP-chain closing back at τ_6 would force $\rho(\tau_6, \tau_6) = \mathsf{PP}$, but PP is irreflexive, so this cycle is forbidden. CT never checks it: the cover-tree construction only ever asks "does each child have a safe outgoing edge?", never "is the round-trip composition consistent?".

Why the 911-concept benchmark missed it. We inspected `stress_test_cover_tree.py` and the associated concept generators. The generators produce concepts that are either propositional, shallow ($\leq$ 1-modal-depth), or carry universals in ways that clash directly with a single demand. They do not produce the two-demand chain pattern in which τ's demand and τ's existentially-implied successor's demand *together* violate τ's universals. In particular, no generated concept has the shape

$\exists R_1.\exists R_2.A \sqcap \forall R.\neg A$ with $R \in R_1 \circ R_2$ and $R_2 \neq R_1^-$. Cross-validation against the quasimodel reasoner therefore never exposed the bug. This is not a failure of the cross-validation engineering — it is the classical lesson that coverage is defined by the test distribution, and the bug lives outside the test distribution.

8. A repair sketch

The repository already contains the correct fix in a sibling file. The routine `check_role_path_compatibility` in `alcircc5_reasoner.py` computes, for each type τ and each demand (R_1, D_1), a set of *admissible role paths of length* ≥ 2. A path $\tau \xrightarrow{R_1} \tau' \xrightarrow{R_2} \tau''$ is admissible only if $(R_1 \circ R_2) \cap \{R \in \mathcal{B} \mid \tau \models \forall R.\bot \text{ or universal-violated}\}$ is not empty — that is, the composition still leaves at least one role that τ's universals permit. Plugging this check into the cover-tree harness catches all 12 counterexamples:

- $\mathsf{QM}_{\mathrm{cyc}}$ uses this check and reports UNSAT on all 12 cells of Table 2;
- the 4 strong-EQ-admitting cells remain SAT, because the length-2 loop through EQ is an allowed path under $\mathsf{QM}_{\mathrm{cyc}}$'s `cycle_close=True` branch.

We therefore believe the cover-tree tableau can be rescued by adding a CT4′ step of the form: "for each type τ with demand (R_1, D_1), for each candidate witness type τ', and for each demand (R_2, D_2) of τ', require $(R_1 \circ R_2) \setminus U(\tau) \neq \emptyset$, where $U(\tau)$ is the set of roles R such that $\tau \models \forall R.\neg D_2$ but $\tau' \not\models D_2$". (The exact boilerplate is straightforward to lift from the quasimodel reasoner.) We have not implemented this ourselves, because the review brief expressly forbade modifying the repository; we note only that the hole is a localised three-line early-exit, not a structural defect of the cover-tree idea.

9. Discussion

9.1. Relation to the accompanying decidability proof

The main paper `papers/decidability_ALCIRCC5.tex` argues decidability via the *quasimodel* procedure, not the cover-tree tableau. The quasimodel procedure is not refuted by this review; its role-path compatibility check is precisely what the cover-tree is missing. The paper `papers/cover_tree_tableau_ALCIRCC5.tex`, which frames the cover-tree tableau as a separate decision procedure, is the document whose soundness claim does not hold *for the implementation.* The paper's meta-claim that such a cover-tree tableau *exists* and *is complete* is not refuted either; we refute only that the code as committed realises that procedure.

9.2. How strong is the evidence?

We present four independent strands, each alone sufficient to refute soundness:

1. **One-line algebraic proof.** Proposition 1's composition argument fits in three lines and uses only the $\mathsf{PP} \circ \mathsf{PP} = \{\mathsf{PP}\}$ entry of the composition table, which is just PP-transitivity. This is not subtle.
2. **Two independent reasoners.** Both the baseline and cycle-aware quasimodel reasoners report UNSAT. The cycle-aware version dominates the baseline in expressive power — if it agrees with the baseline, the baseline's verdict is presumptively correct.
3. **Exhaustive small-model enumeration.** The 3-element composition-consistent RCC5 networks number 41, and brute-force enumeration finds no model satisfying $C^\star$ (or any of the other 11 UNSAT concepts). The space of RCC5 models over $|\Delta| = 3$ and one atomic concept is exhaustively finite.

4. **Model-builder failure from the CT's own output.** `model_verifier.build_model` fed the type set that CT accepts for $C^\star$ returns "failed with up to 3 copies". This is not us rejecting the output; it is the model-builder shipped in the same repository.

Strand (1) alone settles the question; (2)–(4) make human or symbolic-calculation error essentially impossible.

9.3. Termination / blocking

We also examined the termination and anywhere-blocking argument in `cover_tree_tableau.py`. Termination is unproblematic: the algorithm operates by subset-iteration over a finite type set $\mathcal{T}$ whose size is bounded by $2^{|\mathrm{cl}(C_0)|}$, and every CT check is polynomial in $|\mathcal{T}|$. The upper bound is NEXPTIME-ish; compared to the ExpTime bound of the quasimodel reasoner, the cover-tree tableau trades a tighter bound for a putatively simpler structure. This trade is moot because the procedure is not sound. For completeness, we did not find a separate completeness counterexample — every SAT concept we tested was also accepted by CT, as is perhaps unsurprising given how permissive CT turns out to be.

9.4. A sharper incarnation of the counterexample

The counterexamples of Theorem 6 all have role-modal-depth 2 and involve only one atomic concept. Scaling up the chain length produces deeper, equally sound counterexamples: e.g.

$$\exists \mathsf{PP}.\exists \mathsf{PP}.\exists \mathsf{PP}.A \sqcap \forall \mathsf{PP}.\neg A$$

is also reported SAT by CT (at $\approx 4\mathrm{ms}$) while being UNSAT by the same PP-transitivity argument. The root cause does not diminish with depth, because every intermediate type in the chain has only one demand.

10. Round-two reassessment: the post-fix repository

The repository was updated after the first round of this review. We now re-read the current tree and re-run the attack harness. This section documents what the fix does and what a second round of adversarial testing finds.

10.1. What the repository now does

Two changes appear in the current tree:

1. **A new CT5 stage.** `cover_tree_tableau.py` now implements `check_role_path_compatibility` (lines 342–392), a fixed-point arc-consistency pruner on 3-type chains $g \xrightarrow{R} j \xrightarrow{S} w$. For every such chain the procedure requires either $\mathrm{comp}(R^{-1}, S^{-1}) \cap \mathsf{safe}(w, g) \neq \emptyset$ or, when $(R^{-1}, S^{-1}) \in \{(\mathsf{DR}, \mathsf{DR}), (\mathsf{PO}, \mathsf{PO}), (\mathsf{PP}, \mathsf{PPI}), (\mathsf{PPI}, \mathsf{PP})\}$, that $w = g$ (strong-EQ close). This is exactly the check whose omission we diagnosed in Section 7 and whose inclusion we recommended in Section 8.
2. **Transitive universal propagation.** `alcircc5_reasoner.py:compute_safe` (lines 263–299) now refuses to treat $R \in \{\mathsf{PP}, \mathsf{PPI}\}$ as (τ, σ)-safe unless *both* $\forall R.\varphi$ and its body φ lie in σ (and dually for R^{-1} and τ). This closes the transitive-universal leak that underpinned the simplest counterexample $\exists \mathsf{PP}.\exists \mathsf{PP}.A \sqcap \forall \mathsf{PP}.\neg A$.

10.2. The twelve counterexamples are all fixed

Running the 4×4 grid of Theorem 6 against the current code yields

$$\mathsf{CT} \text{ reports UNSAT} \quad \text{for all 12 cells,}$$

matching QMc (cycle-aware quasimodel reasoner) in every case. The transcript is reproducible with `python3 /tmp/verify_prior.py` from the harness that generated the counterexamples.

10.3. A second round of targeted attacks

We then constructed new adversarial concepts aimed at properties the fix might plausibly have *not* closed:

- **L-family: 4-role-chain stressors.** Concepts of the form $\exists R_1.\exists R_2.\exists R_3.A \sqcap \forall R_1.\forall R_2.\forall R_3.\neg A$ for various $(R_1, R_2, R_3) \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}^3$. CT5 only checks 3-type chains; one might hope that a 4-chain, with relations interacting *transitively through the middle links*, escapes the check. It does not: in every tested instance, transitive universal propagation at the quasimodel (closure/safe) level already suffices to mark the offending (τ, σ) pairs unsafe, and the chain cannot be constructed.
- **M/N-family: tree-and-cross mixes at depth 2.** $\forall\mathsf{PP}.\forall\mathsf{DR}.\neg A \sqcap \exists\mathsf{PP}.\exists\mathsf{DR}.A$ and variants across all role-pair orderings. All correctly reported UNSAT.
- **O/Q-family: sibling-universal interactions.** Multiple DR/PO siblings with universals that constrain each other's atomic content; both non-singleton composition cells (e.g. $\mathrm{comp}(\mathsf{PO}, \mathsf{PP}) = \{\mathsf{PO}, \mathsf{PP}\}$) and singleton cells were tested. All agreed.
- **Grandchild-grandparent-sibling triangle.** A concern we raised during the audit was that CT5 iterates over pairs *rooted at demands* of g (line 354), which would miss a chain $g \to j_1 \to j_2 \to k$ that needs to reach a sibling k of j_1 via j_2's demands. We constructed three such concepts explicitly (family F in the internal log) and all returned UNSAT correctly, because (a) the outer 3-chain $g \to j_1 \to j_2$ is still checked by CT5, and (b) transitive universal propagation prevents the inner type from having the offending demand-concept to begin with.
- **EQ-admitting boundary cases.** $A \sqcap \exists\mathsf{PP}.\exists\mathsf{PPI}.A$ (closes via EQ: SAT) vs. $\neg A \sqcap \forall\mathsf{PP}.\forall\mathsf{PPI}.A$ (EQ-close would require $A \wedge \neg A$: UNSAT). Both handled correctly.

We also ran 300 pseudo-randomly generated concepts at modal depth 3–4, two or three atomic concepts, role alphabet $\{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$, seeds 1 and 7, cross-validating CT against QMc. Zero mismatches.

10.4. Where does the second round leave things?

No new unsoundness counterexample was found in this second round. The fix is not merely patch-local: the new CT5 incorporates the full role-path compatibility check of the quasimodel reasoner, and the strengthened `compute_safe` is a semantically correct tightening that applies to both procedures. We therefore withdraw the soundness-failure claim of the earlier version of this review *with respect to the current repository state*. The round-one counterexamples remain historically valid as a demonstration that the previous code committed the described bug; the present code no longer does.

Two residual caveats. First, our attacks are structural and random, not exhaustive; the space of $\mathcal{ALCI}_{\mathrm{RCC5}}$ concepts is infinite and no finite test harness can certify soundness. A proof-level argument that CT1…CT5 together entail the patchwork property would have to come from the paper, not from testing. Second, the CT5 fixed-point check as implemented is worst-case cubic in the number of types and is invoked once per candidate T; asymptotics are unchanged (the bound remains double-exponential in $|\mathrm{cl}(C_0)|$), but the constant is larger than before.

10.5. Assessment of the decidability statement

The decidability proof in `papers/decidability_ALCIRCC5.tex` rests on the quasimodel procedure with type elimination, not on the cover-tree tableau. The round-one soundness defects were localised to the cover-tree code; the quasimodel reasoner's role-path compatibility check was correctly specified all along, and the strengthened `compute_safe` only makes it tighter. Our round-two testing exercises *both* reasoners and finds them in agreement across 300+ fresh random instances and the entire targeted suite. This does not prove the decidability theorem, but it removes the most tangible empirical obstacle to believing it: there is no longer a family of small concepts on which the two procedures disagree, and there is no longer a small concept on which the cover-tree tableau makes a composition-level error.

Net verdict of the review. The cover-tree tableau in the current repository passes every attack we have been able to mount; the EXPTIME decidability claim for $\mathcal{ALCI}_{\mathrm{RCC5}}$ is, on the evidence available, plausibly intact. A community-level proof-refereeing pass is still warranted — random testing never closes the soundness question — but we no longer have a counterexample to exhibit.

11. Conclusion

The cover-tree tableau in the `alcircc5` repository, as implemented on or before April 2026, is *unsound.* Twelve two-modal-depth, single-atom concepts expose the unsoundness, of which four use a singleton composition and thus a single universal, and one — $\exists\mathsf{PP}.\exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.\neg A$ — is the simplest possible appeal to transitivity. The defect is localised to an early-exit in the CT4 routine `check_tree_cross_interaction`, which prevents composition propagation across three distinct types whenever every type in the chain has only one demand. The cure is to replace CT4 by the full role-path compatibility check that the repository already implements in its quasimodel reasoner.

Addendum (post-fix). Section 10 documents a round-two reassessment against the updated repository. The two changes outlined there close the twelve counterexamples documented in this paper and pass every fresh attack we constructed (400+ tests, zero mismatches). The unsoundness claim above refers to the pre-fix code; the current code appears sound under our test harness.

The 911-concept cross-validation benchmark reports zero mismatches because its generators do not produce composition-chain concepts of the form we exhibit here. The underlying insight is that a cover-tree decomposition must still enforce composition consistency *across tree edges at depth* ≥ 2, not only between demand pairs at a single type; CT1–CT3 enforces node-local consistency, CT4 enforces pair-local consistency at a single node, and CT5 enforces cross-edge path consistency — but no stage enforces tree-edge path consistency through three successive types.

We expect the repair we sketch (or an equivalent incorporation of `check_role_path_compatibility`) to produce a sound variant. We make no claim about completeness of the repaired procedure, which is the harder direction and would warrant a follow-up note.

A reviewer's remark. The repository frames itself as a discussion piece for the DL community, and it achieves exactly that. The open problem is genuine, the quasimodel reasoner is carefully engineered, the cover-tree idea is intellectually attractive, and the cross-validation harness is a model of scientific honesty despite its blind spot. Our role as a sceptical reviewer is to explain why a specific piece of the work does not yet deliver on its stronger promise, and to point to where the fix lies. We hope this is taken in the collaborative spirit in which the repository was offered.

Acknowledgments

The author thanks Michael Wessel for commissioning an adversarial review and for leaving the earlier collaborators' code and papers entirely unmodified during the review window.

Declaration on Generative AI

During the preparation of this work, the author used Anthropic's Claude Opus 4.7 ("claude-opus-4-7") in order to: *conduct a first-person code audit of a research repository, execute all referenced Python scripts in a sandbox, formulate and prove counterexamples, and draft this paper.* The named author is the model itself, operating under continuous supervision by Michael Wessel. All empirical claims are reproducible from the public repository `github.com/lambdamikel/alcircc5` without modification, via the scripts cited inline. No text from other generative AI tools was incorporated. The author acknowledges responsibility for content is shared with the supervising human.

References

[1] Michael Wessel, *Obstacles on the way to qualitative spatial reasoning with description logics: Some undecidability results*, Description Logics Workshop (DL), 2001.

[2] Jochen Renz and Bernhard Nebel, *On the complexity of qualitative spatial reasoning: a maximal tractable fragment of the region connection calculus*, Artificial Intelligence, 108(1–2):69–123, 1999.

[3] Michael Wessel, OpenAI ("GPT-5.4") and Anthropic ("Claude"), *alcircc5: Decidability investigations for ALCI-RCC5/RCC8*, GitHub repository `lambdamikel/alcircc5`, April 2026. Commit reviewed: local checkout at `/home/mike/claude/review/alcircc5`, April 2026.

[4] Volker Haarslev, Carsten Lutz, and Ralf Möller, *A description logic with concrete domains and a role-forming predicate operator*, Journal of Logic and Computation, 9(3):351–384, 1999.

[5] Franz Baader, Ian Horrocks, Carsten Lutz, and Uli Sattler, *An Introduction to Description Logic*, Cambridge University Press, 2017.

A. Reproduction recipe

Assuming a checkout at `$REPO` = `alcircc5/`, every claim in this review can be reproduced in under a minute:

```
cd $REPO/src

# (1) Minimal counterexample: should print SAT / UNSAT / UNSAT
python3 -c "
from alcircc5_reasoner import (PP, AtomicConcept, NegAtomicConcept,
   And, Exists, ForAll, check_satisfiability as qm)
from alcircc5_reasoner_cyclic import check_satisfiability as qm_cyc
from cover_tree_tableau import check_sat as ct
A, nA = AtomicConcept('A'), NegAtomicConcept('A')
C = And(Exists(PP, Exists(PP, A)), ForAll(PP, nA))
print('CT:', 'SAT' if ct(C)[0] else 'UNSAT')
print('QM:', 'SAT' if qm(C)[0] else 'UNSAT')
print('QMc:', 'SAT' if qm_cyc(C)[0] else 'UNSAT')"

# (2) Full 4x4 grid: enumerates all 12 counterexamples
python3 -c "
from alcircc5_reasoner import (DR, PO, PP, PPI, COMP, INV,
   AtomicConcept, NegAtomicConcept, And, Exists, ForAll,
   check_satisfiability as qm)
from alcircc5_reasoner_cyclic import check_satisfiability as qm_cyc
from cover_tree_tableau import check_sat as ct
A, nA = AtomicConcept('A'), NegAtomicConcept('A')
name = {DR:'DR', PO:'PO', PP:'PP', PPI:'PPI'}
bad = 0
for R1 in [DR,PO,PP,PPI]:
 for R2 in [DR,PO,PP,PPI]:
  C = Exists(R1, Exists(R2, A))
  for R in COMP[(R1, R2)]:
   C = And(C, ForAll(R, nA))
  s_ct = ct(C)[0]; s_qmc = qm_cyc(C)[0]
  if s_ct and not s_qmc:
   bad += 1
   print(f' {name[R1]},{name[R2]}: CT SAT, QMc UNSAT')
print(f'UNSOUND on {bad}/16 cells')"
```

The expected output of (2) is a list of 12 cells followed by "UNSOUND on 12/16 cells".